

Project 1: Neural Network and CNN for MNIST Classification

Name: Yu Yuxuan (余宇轩) Student ID: 22300680039

May 21, 2026

1. Introduction

This project studies handwritten digit classification on MNIST using neural networks implemented with NumPy. The implementation does not use deep learning libraries for the required operators. I completed the forward and backward propagation of linear layers, the softmax cross-entropy loss, a 2D convolution operator, an MLP baseline, and a simple CNN. I also explored two additional directions: optimization with momentum and L2 regularization. To make the comparison more interpretable, I added confusion matrices, confident-error examples, MLP weight visualization, and CNN kernel visualization.

Code repository: TODO

Trained model/checkpoint link: TODO

2. Implementation

2.1 Core Operators

The core implementation is in `codes/mynn/op.py`.

- `Linear.forward` computes $XW+b$.
- `Linear.backward` computes gradients for (W) , (b) , and the previous layer.
- `MultiCrossEntropyLoss` combines softmax and multi-class cross-entropy. The backward pass uses $((p-y)/N)$, where (p) is the softmax probability and (N) is the batch size.
- `conv2D` implements a NumPy-based 2D convolution layer for inputs shaped as `[batch, channels, height, width]`. The forward pass explicitly unfolds each receptive-field patch into an `im2col` matrix with loops, and the backward pass scatters gradients back to the padded input while computing gradients for kernels and bias.

2.2 Models

The MLP baseline uses:

- input dimension: 784
- hidden layer: 600 ReLU units
- output dimension: 10

The CNN uses:

- one convolution layer with 8 kernels, kernel size (3), stride 1, and padding 1
- ReLU activation

- flatten layer
- linear classifier to 10 classes

The CNN is intentionally simple, so the comparison focuses on the effect of convolutional local structure rather than model complexity.

3. Experimental Setup

The dataset was split into 50,000 training samples and 10,000 validation samples from the provided MNIST training set. The official MNIST test set with 10,000 samples was used only for final evaluation.

Unless otherwise stated:

- batch size: 64
- epochs: 5
- scheduler: MultiStepLR with milestones 800, 2400, and 4000, gamma 0.5
- random seed: 309
- validation metric: accuracy

4. Results

Experiment	Main Setting	Best Validation Accuracy	Test Accuracy	Checkpoint
MLP baseline	SGD, no L2	94.48%	94.77%	codes/ best_models/ mlp_baseline/best_model.pickle
CNN baseline	SGD, no L2	96.54%	96.83%	codes/ best_models/ cnn_baseline/best_model.pickle
CNN + Momentum	Momentum, (), lr 0.03	97.80%	97.80%	codes/ best_models/ cnn_momentum/best_model.pickle
CNN + L2	SGD, L2 (10^{-4})	96.55%	96.82%	codes/ best_models/ cnn_l2/best_model.pickle

5. MLP Baseline

The MLP reaches 94.77% test accuracy. This is a reasonable baseline because the model can learn global pixel patterns from flattened images. However, flattening removes the spatial structure of the image. Neighboring pixels are treated like arbitrary independent input features, so the model must learn local visual patterns from data rather than using a built-in local inductive bias.

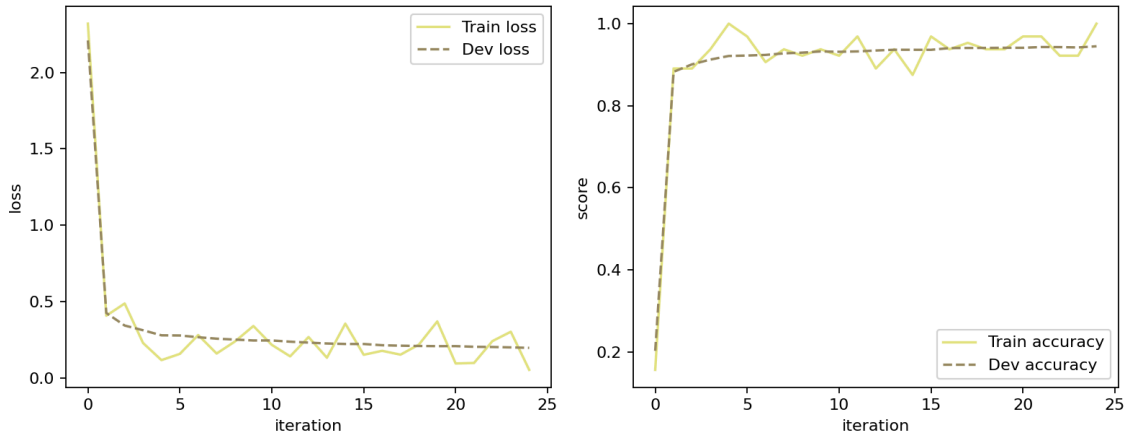


Figure 1: MLP learning curve

6. CNN Model and Comparison

The CNN reaches 96.83% test accuracy, outperforming the MLP by 2.06 percentage points on the test set. The improvement is consistent with the expected advantage of convolution for image classification:

- convolution uses local receptive fields, which match the local stroke patterns in handwritten digits;
- the same kernel is shared across spatial locations, reducing the number of parameters and improving sample efficiency;
- preserving image layout before the classifier makes the model better suited to digit shapes.

The CNN used here is still a small model with only one convolution layer. Therefore, the result should be interpreted as evidence that even a simple convolutional inductive bias helps on MNIST.

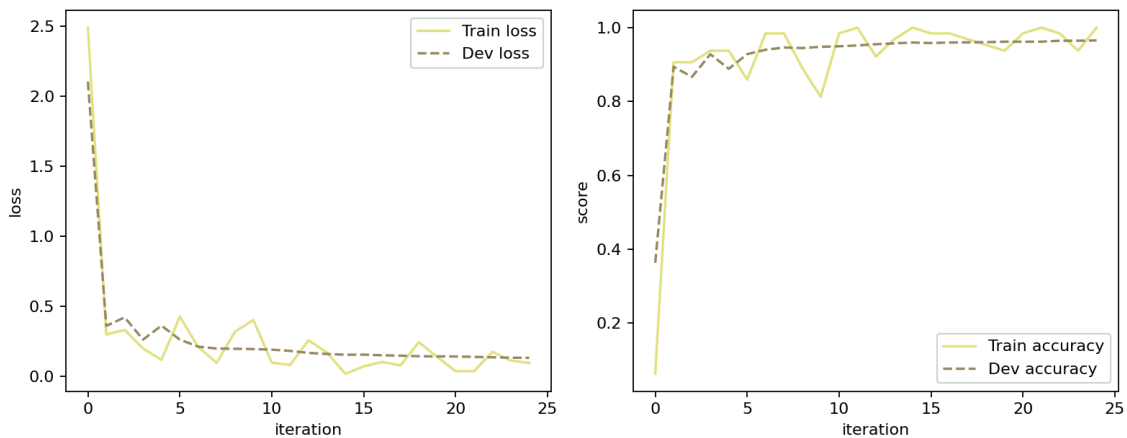


Figure 2: CNN learning curve

7. Additional Direction 1: Optimization

I compared the CNN baseline with a momentum optimizer. The momentum experiment used $\eta = 0.03$ and learning rate 0.03. Momentum improves both convergence and final accuracy: the best validation accuracy increases from 96.54% to 97.80%, and the test accuracy increases from 96.83% to 97.80%.

This suggests that momentum helps the optimizer move more consistently through the loss landscape and reduces the effect of noisy mini-batch gradients. I used a smaller learning rate than the SGD baseline because momentum accumulates velocity and can otherwise make updates too aggressive.

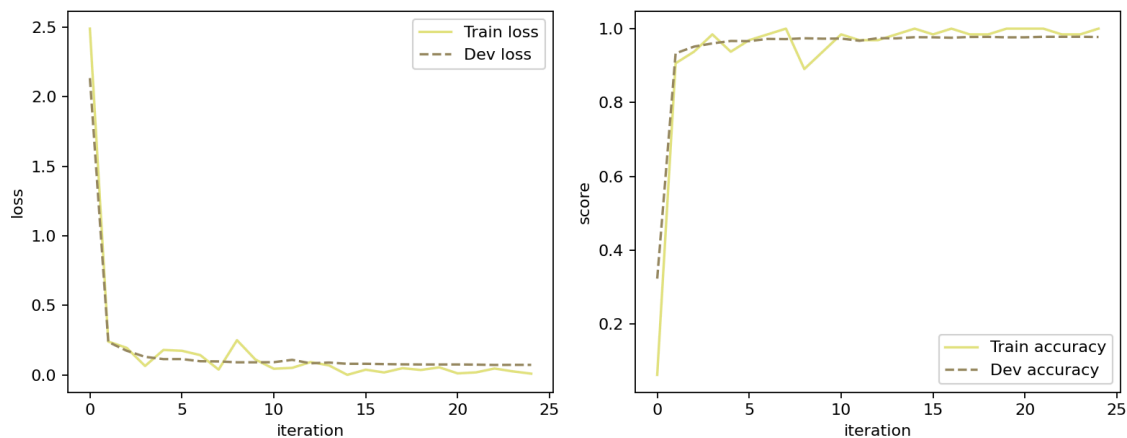


Figure 3: CNN momentum learning curve

8. Additional Direction 2: Regularization

I also tested L2 regularization with coefficient (10^{-4}) on the CNN. The result is almost the same as the unregularized CNN: validation accuracy changes from 96.54% to 96.55%, and test accuracy changes from 96.83% to 96.82%.

This indicates that the simple CNN did not suffer from severe overfitting under the current training setup. Since the model is small and trained for only 5 epochs, the regularization effect is limited. A larger model or longer training may show a stronger difference.

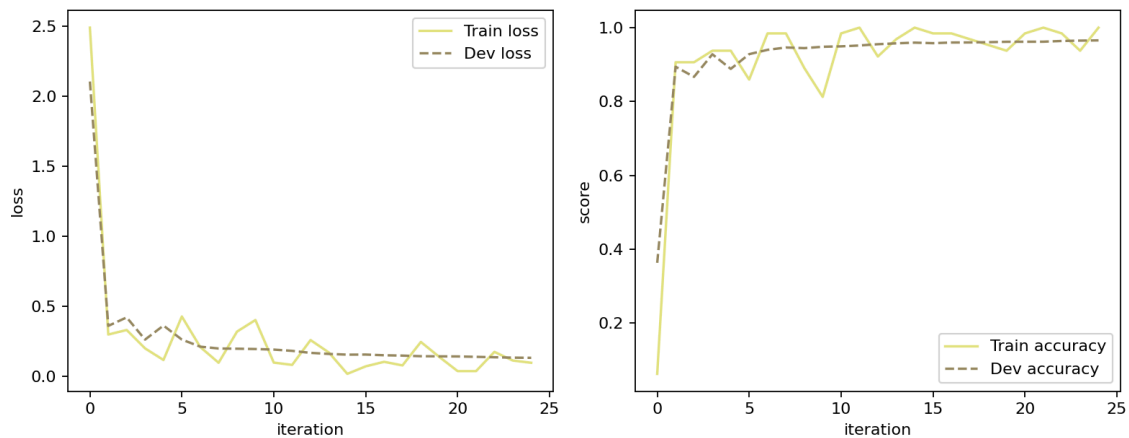


Figure 4: CNN L2 learning curve

9. Detailed Visualization and Error Analysis

The confusion matrices show that CNN improves the overall diagonal dominance compared with MLP. The MLP baseline mainly confuses 9 as 4, 4 as 9, and 7 as 9 or 2. The CNN reduces many of these mistakes, but 7 and 2 remain difficult. With momentum, the same CNN reaches the best test accuracy, and the remaining largest confusions are still visually plausible digit pairs such as 7/2, 9/4, and 4/9.

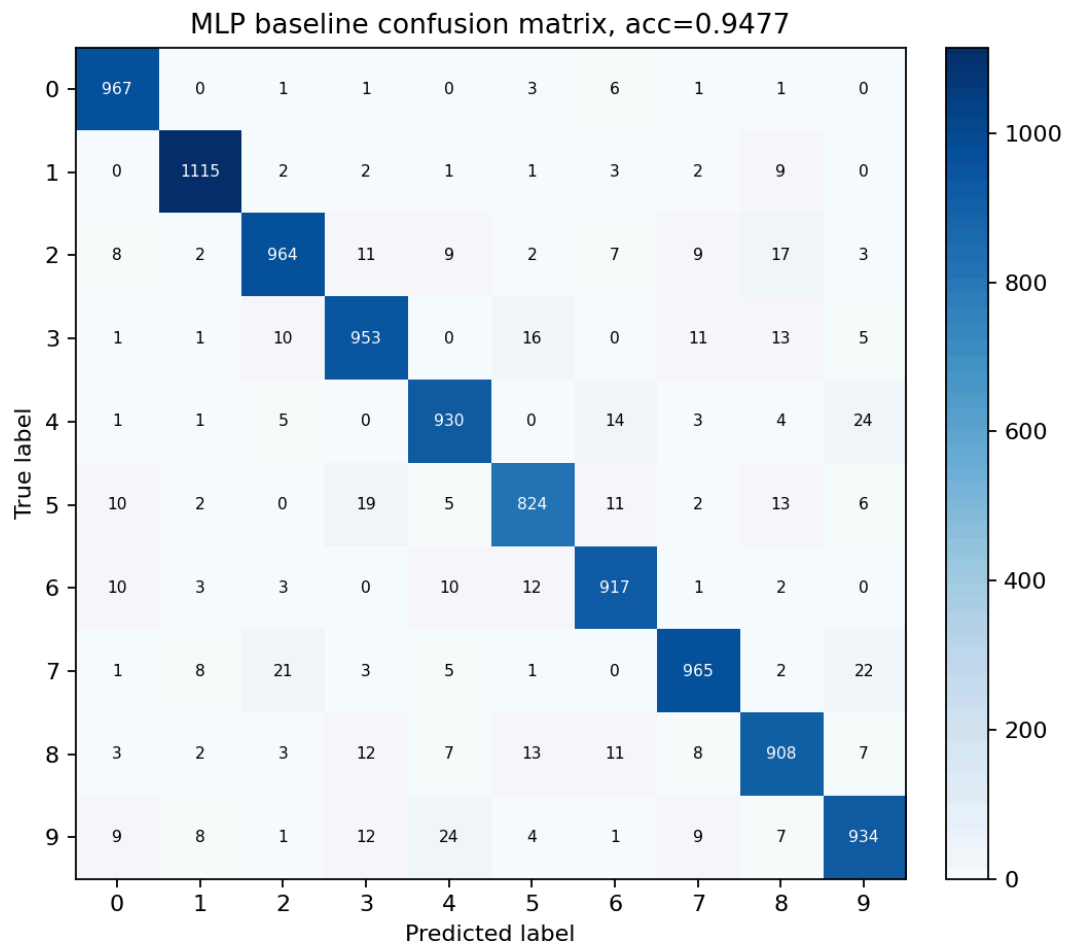


Figure 5: MLP confusion matrix

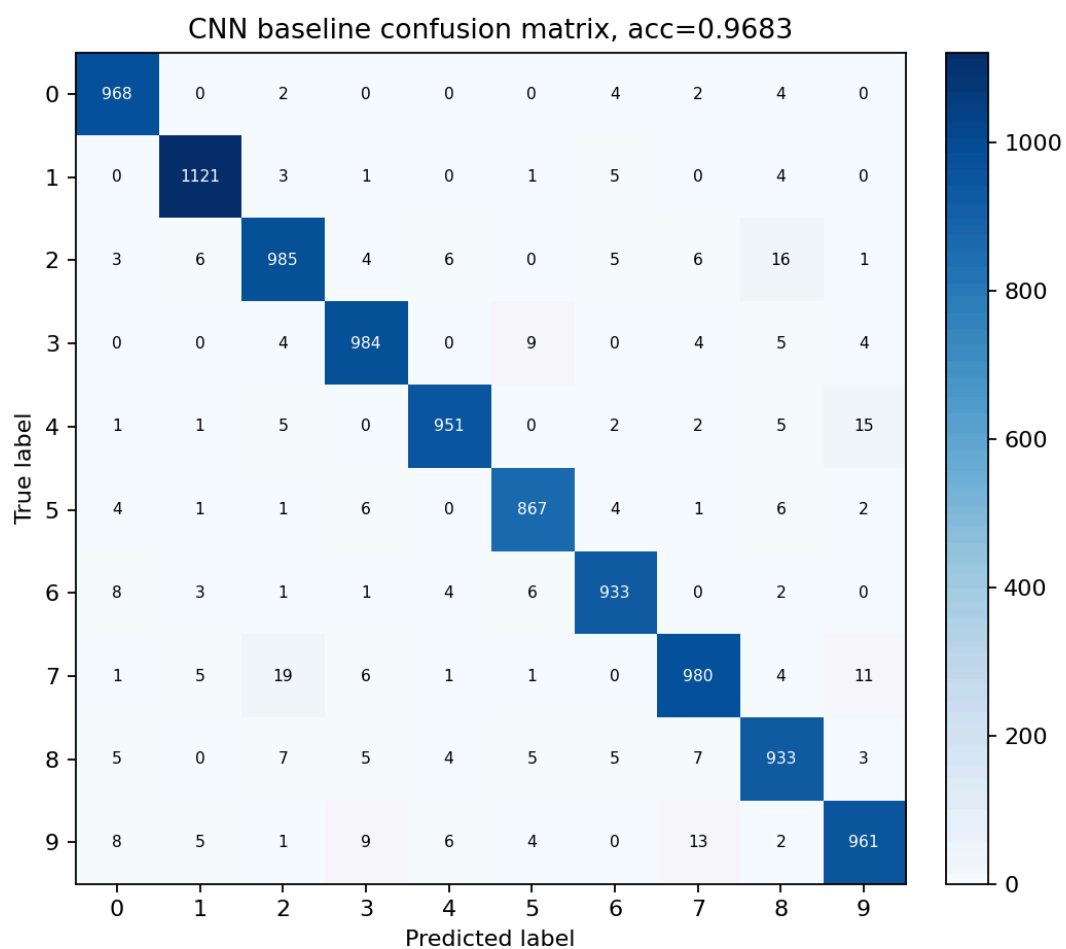


Figure 6: CNN confusion matrix

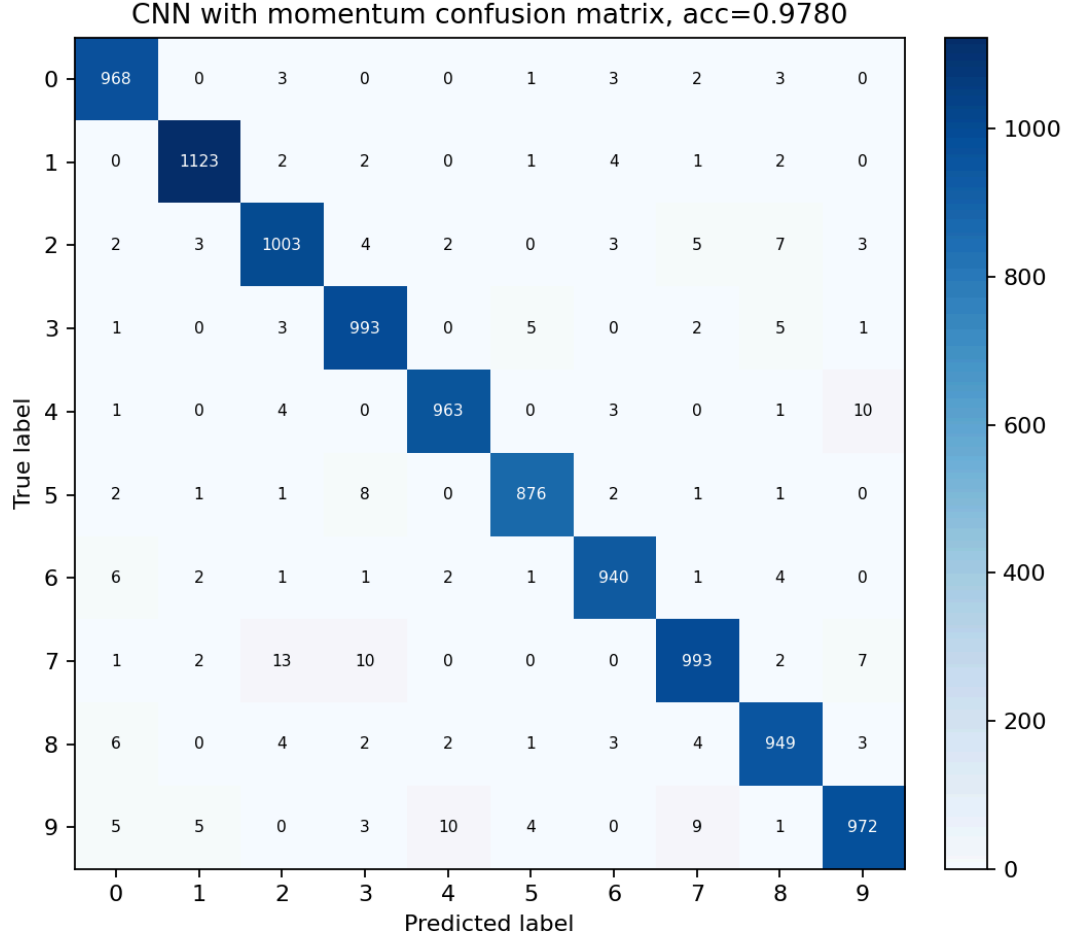


Figure 7: CNN momentum confusion matrix

The confident-error examples are useful because they reveal failure cases where the model is not merely uncertain, but confidently wrong. Many of these examples are ambiguous even to a human reader: some 7s have strong curved strokes similar to 2, and some 4/9 samples share a closed upper loop.

MLP baseline: confident mistakes

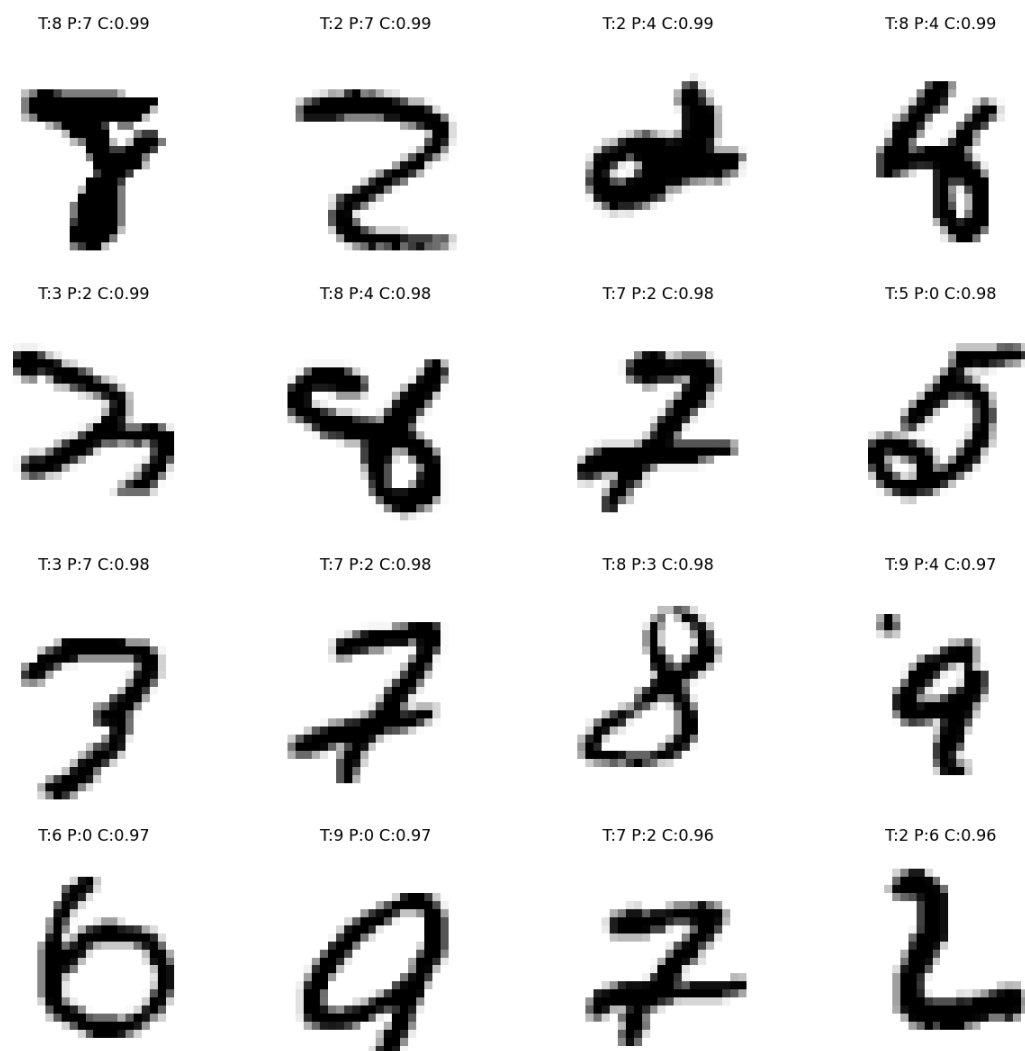


Figure 8: MLP confident errors

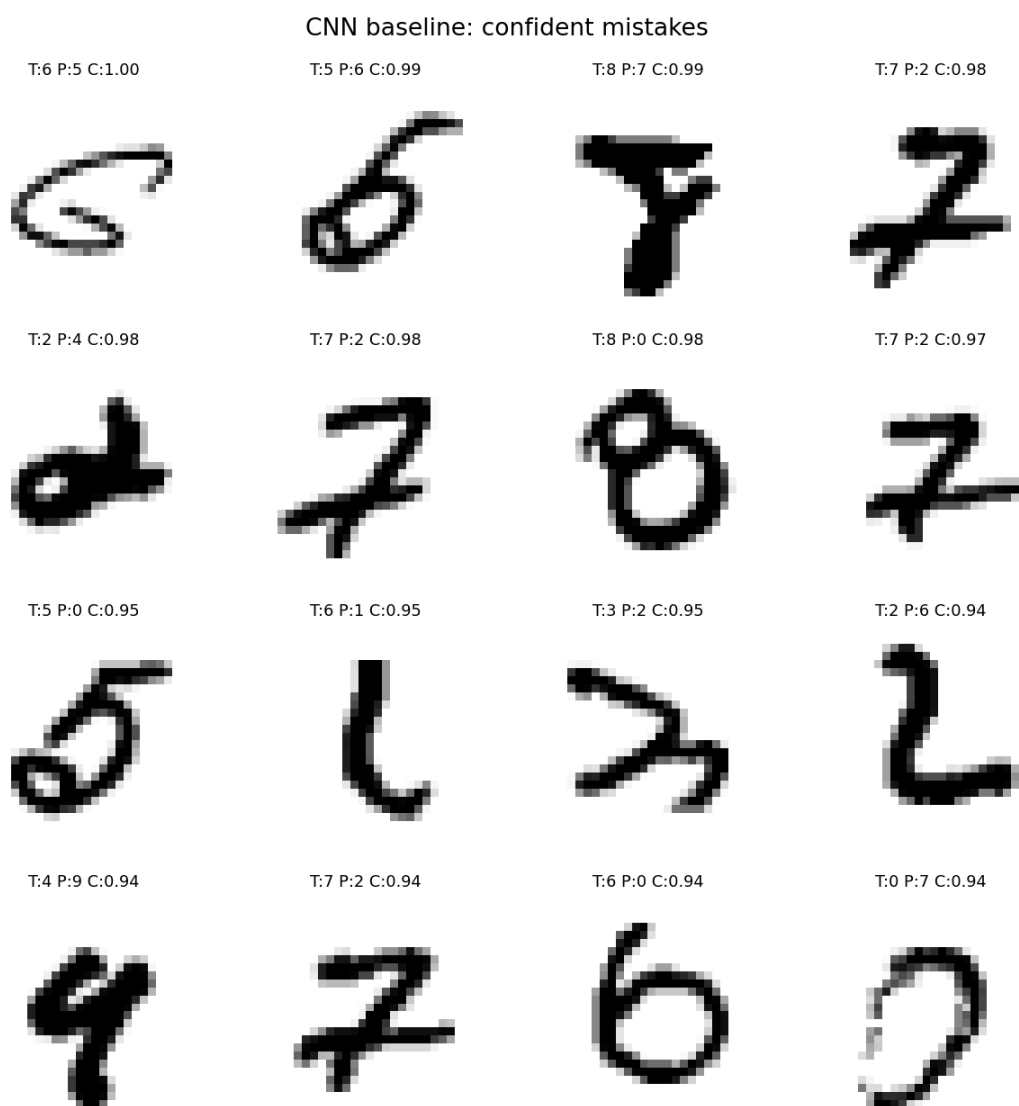


Figure 9: CNN confident errors

The first-layer MLP weights can be reshaped into 28 by 28 images. They look like coarse digit templates or stroke detectors, but each hidden unit is connected to the full image. In contrast, the CNN kernels are small local filters. Even in this simple one-layer CNN, different kernels emphasize different local directions and contrast patterns.

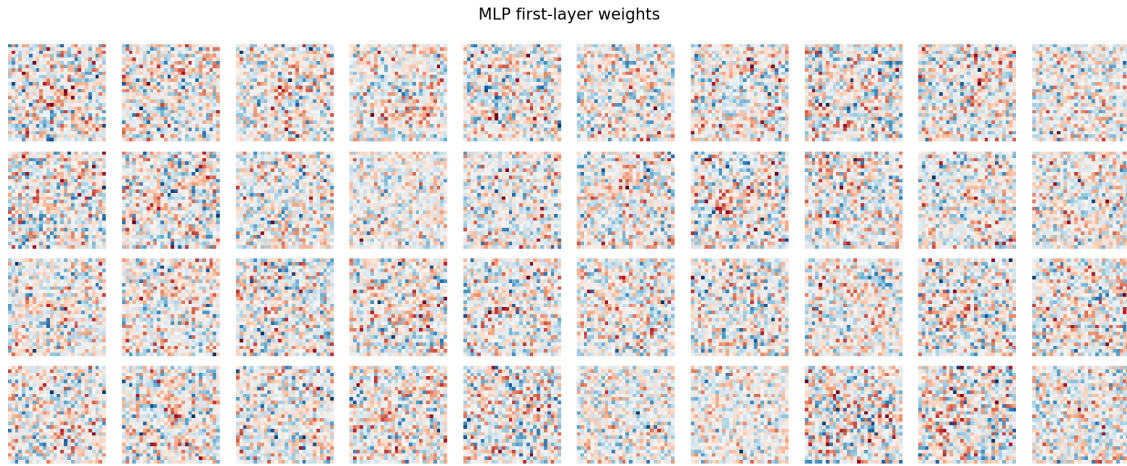


Figure 10: MLP first-layer weights

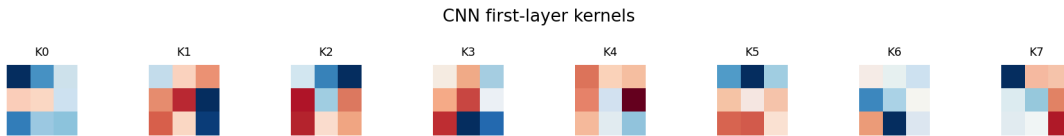


Figure 11: CNN first-layer kernels

10. Discussion

The main result is that the CNN performs better than the MLP on MNIST. The reason is not only that the CNN has convolution operators, but that those operators encode a useful assumption: local patterns can appear at different positions in an image. This is exactly the case for handwritten digits, where strokes and edges are local and spatially organized.

Momentum is the most useful additional modification in this experiment. It improves the CNN test accuracy from 96.83% to 97.80%. L2 regularization has little effect here, probably because the model is not large enough to overfit heavily.

The remaining hard cases are not random. The confusion matrices and misclassified samples show that the model still struggles with visually similar digits such as 7/2, 4/9, 9/4, and 5/3. This suggests that the remaining errors are partly caused by ambiguous handwriting rather than only by insufficient model capacity.

11. How to Reproduce

From the codes directory:

```
python .\test_train.py --model mlp --optimizer sgd --scheduler multistep --
weight-decay 0 --num-epochs 5 --batch-size 64 --log-iters 200 --run-name
mlp_baseline
python .\test_train.py --model cnn --optimizer sgd --scheduler multistep --
weight-decay 0 --num-epochs 5 --batch-size 64 --log-iters 200 --run-name
cnn_baseline
```

```
python .\test_train.py --model cnn --optimizer momentum --scheduler multistep --  
weight-decay 0 --lr 0.03 --num-epochs 5 --batch-size 64 --log-iters 200 --run-  
name cnn_momentum  
python .\test_train.py --model cnn --optimizer sgd --scheduler multistep --  
weight-decay 0.0001 --num-epochs 5 --batch-size 64 --log-iters 200 --run-name  
cnn_l2
```

Final test evaluation:

```
python .\test_model.py --model-path .\best_models\mlp_baseline\best_model.pickle  
python .\test_model.py --model-path .\best_models\cnn_baseline\best_model.pickle  
python .\test_model.py --model-path .\best_models\cnn_momentum\best_model.pickle  
python .\test_model.py --model-path .\best_models\cnn_l2\best_model.pickle
```

Visualization:

```
python .\analyze_results.py
```